

Assessed Problem Set 1

Machine Learning EFDS 2025

Group C

Nov 5, 2025

Problem 1. No Free Lunch Theorem and Manifold Hypothesis for Natural Language

1. We consider all $26 \times 26 = 676$ possible two-letter combinations in the English alphabet. Using Python, we plot each pair where the x -axis represents the first letter and the y -axis represents the second letter. Points corresponding to real two-letter English words (based on the provided Wikipedia list) are marked in green, while all others are shown in black.

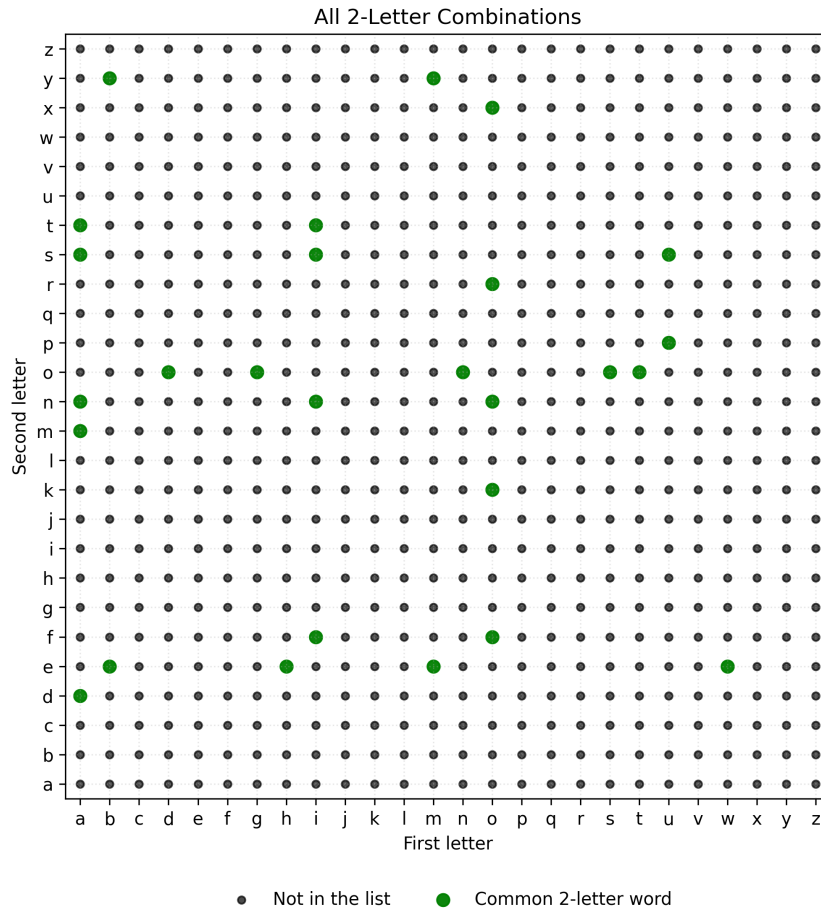


Figure 1: Two-letter grid

Figure 1 visualises these combinations; the sparse pattern of green points shows that only a small fraction of letter pairs form valid English words.

The Python script used to generate this figure also computes the proportion of valid two-letter words.

output:

common words: 27 / 676

Fraction: 0.039941

Percentage: 3.99%

Only 27 out of 676 possible two-letter combinations correspond to actual English words. Thus, approximately 3.99% of all two-letter combinations form common English words, highlighting the sparsity of meaningful patterns in the space of possible combinations.

2. We draw random 8-letter strings uniformly from $\{a, \dots, z\}^8$, seeding the random number generator by the group letter position ($C \rightarrow \text{seed} = 3$). An 8-letter draw is a success if it appears in `eight-letter-words.csv`. With $|D| = 983$, the per-draw success probability is

$$p = \frac{|D|}{26^8} = \frac{983}{26^8} \approx 4.707 \times 10^{-9}, \quad \mathbb{E}[\text{draws}] = \frac{1}{p} \approx 2.12 \times 10^8.$$

A geometric model with $\text{seed} = 3$ yielded

$$\text{draws until first word} = \mathbf{23,371,384}.$$

To gauge typicality across seeds A–Z, one geometric sample per seed gave:

$$\text{min} = 9,919,607 \quad \text{median} = 141,131,939 \quad \text{max} = 807,017,110 \quad \text{IQR} = 59,262,616\text{--}306,408,767$$

Our seed’s result lies *below* the IQR, so it is **particular** (on the low side) but still plausible for a high-variance geometric distribution.

We also ran a literal simulation capped at 5,000,000 random draws. Since the probability of observing at least one valid 8-letter English word within that limit is only

$$\Pr(\text{hit within 5M}) = 1 - (1 - p)^{5 \times 10^6} \approx 2.33\%,$$

no hit occurred within the cap, an expected outcome confirming that valid English words are exceedingly sparse in the space of all possible 8-letter combinations.

3. “*To be, or not to be, that is the question*” - contains six two-letter words: *to, be, or, to, be, is*.

(a) With the list of 27 common two-letter words, replacing exactly one position and keeping the other five fixed yields

$$6 \times (27 - 1) = \mathbf{156}$$

single-word replacements (we exclude replacing a word by itself).

(b) Only a few such replacements yield sentences that still read naturally. Examples include:

- $be \rightarrow do$: “To **do**, or not to be, that is the question.”
- $be \rightarrow go$: “To **go**, or not to be, that is the question.”
- second $be \rightarrow do/go$: “To be, or not to **do/go**, that is the question.”
- $or \rightarrow if$: “To be, **if** not to be, that is the question.”
- $is \rightarrow be$ (archaic but meaningful): “that **be** the question.”

Thus, among the **156** possibilities, only a *handful* (on the order of a few) “make sense.” *Interpretation.* The meaning of the sentence is not robust to arbitrary replacement of small function words: most swaps break grammar or semantics. This mirrors Part 1: just as only about 4% of all letter pairs form valid words, only a tiny subset of replacements preserve sensible meaning, reflecting the sparsity of meaningful structures in language.

4. The results across Parts 1-3 illustrate that meaningful linguistic patterns form a minute, highly structured subset within the vast combinatorial space of all possible letter or word combinations. Randomly sampling from this space almost never yields valid or meaningful outcomes.

According to the **No Free Lunch Theorem**, without exploiting structural information about the domain, in this case, the syntax and semantics of English, no search strategy performs better than random guessing. This explains why unstructured random draws so rarely produce valid words or sentences.

The **Manifold Hypothesis** complements this view: meaningful linguistic data lie on a low-dimensional manifold embedded within a much higher-dimensional space of all possible symbol sequences. Learning or generating language thus requires finding and moving along this manifold rather than exploring the space uniformly. Together, these ideas show that both intelligence and learning depend on structure: meaningful outcomes emerge only by leveraging the underlying organization of data, not by chance.

Problem 2. Cross-Validation

1. See Python File
2. Advantages and Disadvantages of a Larger Tuning Parameter k for k -Fold Cross-Validation

Advantages	Disadvantages
Since the model is trained on less data, the estimated error from cross-validation is likely different from the true test error. We can call this difference the bias of the estimate. $\text{Bias}(\hat{E}_{cv}) = \mathbb{E}[\hat{E}_{cv}] - \mathbb{E}[E_{test}]$ Here, $\mathbb{E}[E_{test}]$ is the expected test error on all the data, and $\mathbb{E}[\hat{E}_{cv}]$ is the average error from cross-validation. A larger k means the training set is larger, so it is closer to the whole dataset. As a result, the estimated test error from cross-validation will tend towards the true test error. Therefore, the bias in the estimated test error from cross validation falls as k increases.	The model is validated on $1/k$th of the dataset. As k increases, the amount of data used for validation falls. This increases the variance of the estimated error across folds. The variance of one cross-validation fold's mean loss estimate is: $\text{Var}[\hat{R}_j] = \frac{s_\ell^2}{ D_j } = \frac{ks_\ell^2}{n},$ where s_ℓ^2 is the sample variance of the individual losses for a fold. This comes from the formula for the variance of a sample mean (the average of D_j i.i.d. loss terms). The result assumes that individual losses in the validation fold are independent; if there is covariance between them, it will inflate the variance of the fold's mean loss. Nonetheless, we will assume the variance of one fold's loss estimate increases linearly with k. Furthermore, training sets for different iterations overlap when $k > 2$. This means that models will be trained on nearly the same data so there will be covariance between each estimated validation error. The total variance of the CV estimate is: $\begin{aligned} \text{Var}[\hat{R}_{CV}(k)] &= \frac{1}{k^2} \sum_{j=1}^k \text{Var}[\hat{R}_j] \\ &\quad + \frac{2}{k^2} \sum_{j < l} \text{Cov}(\hat{R}_j, \hat{R}_l) \end{aligned}$ The first term is approximately constant in k (as the variance of one fold's loss estimate increases linearly with k) and the second term increases as k increases. This means that a larger k leads to a higher variance of the estimated test error.
When k is small, the training set is smaller, so models may underfit as they fail to capture the data's structure, leading to higher validation errors. A larger k increases the training data, enabling each model to learn the underlying relationships more effectively. So, the model is less likely to be underfitted.	The model will likely be overfitted if k gets very large as it would include the noise (as it is trained on nearly the whole dataset each time). The model is thus built on including the random variation. Som if there is noise in the data, a larger k can reduce the generalisability of the model.
A larger k is beneficial when the dataset is scarce. It can be harder to establish an underlying model correctly with fewer points, a larger k can more accurately estimate the model's performance (eg. test error). However, this benefit is still limited by noise in the data. If there is no noise, a larger k is ideal as you don't risk overfitting the model.	The model is trained k times. As k increases, runtime can increase. Especially for more complex algorithms, a large k can make them very slow. This increased computational cost forces one to limit k, as they don't have unlimited computational resources.

References: (James et al., 2013), (Pennsylvania State University, 2018)

Problem 3. Cross-Validation for k-NN Classifier

1. A training dataset was generated with the following attributes:

- Class 0 samples: 56
- Class 1 samples: 44

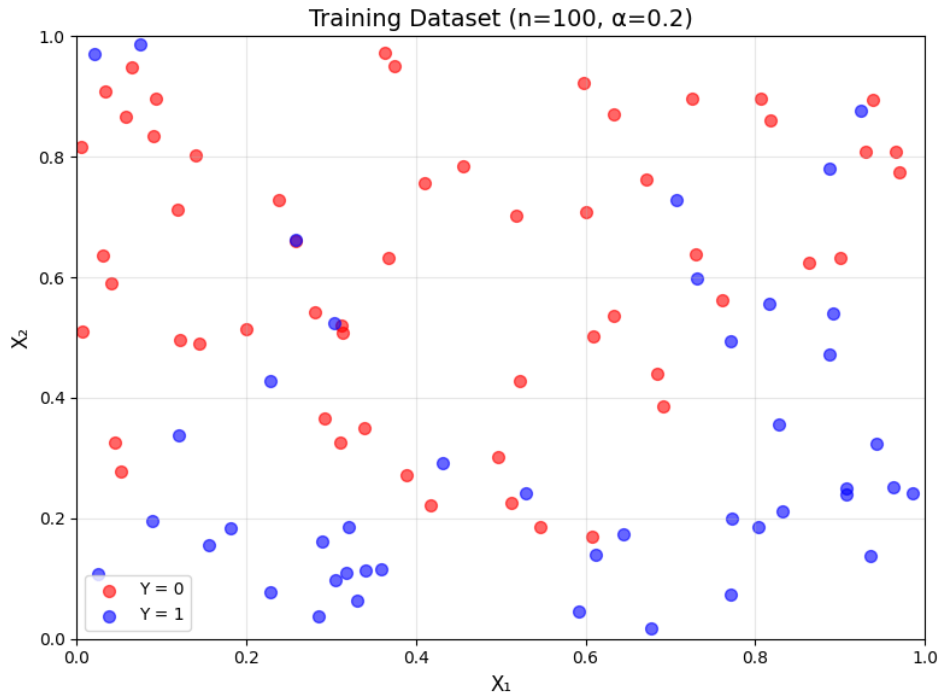


Figure 2: Training dataset with $n = 100$ and $\alpha = 0.2$

The training data shows the expected pattern based on the underlying decision boundary $f(X_1)$ and noise parameter $\alpha = 0.2$:

- Points aren't perfectly separated, showing the presence of label noise
- The noise parameter $\alpha = 0.2$ ensures that even points far from the boundary have at least 20% probability of being assigned to class 1
- The distribution reflects the probabilistic nature of the label generation process:

$$P[Y = 1 \mid X_1, X_2] = \max(\mathbb{1}\{X_2 < f(X_1) - \alpha\}, \alpha)$$

2. Results Summary

Cross-validation errors for each k value:

k	CV Error $\pm$ Std
1	0.3200 ± 0.0682
3	0.2800 ± 0.1082
5	0.1700 ± 0.1128
7	0.1800 ± 0.1128
9	0.1900 ± 0.1470
11	0.2100 ± 0.0586
13	0.2600 ± 0.0974
15	0.2400 ± 0.0804

Best k : 5 (CV error = 0.1700)

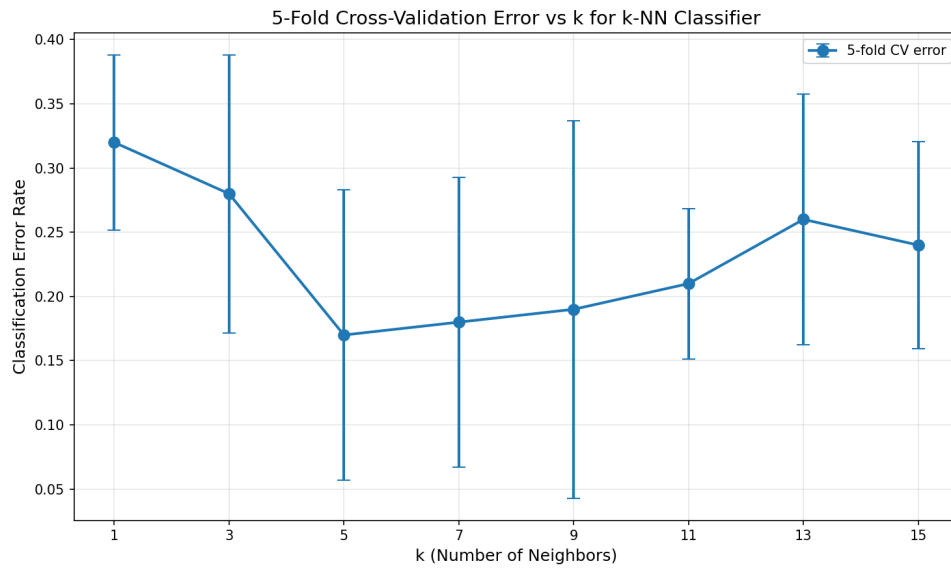


Figure 3: 5-fold cross-validation error vs k for k-NN classifier

Analysis

When k is very small, the model tends to overfit, potentially leading to higher error due to sensitivity to noise in the training data. As for the optimal k range, the CV error typically reaches a minimum at moderate k values where the model balances flexibility and stability.

As k increases beyond the optimal value, the model may underfit, smoothing out important patterns and potentially increasing error. The standard deviation across folds shows the variability in performance estimates which tends to be larger for extreme k values. $k = 5$ represents the sweet spot in the bias-variance tradeoff for this dataset.

3. Yes, the estimates change between runs.

Analysis

- Run 1 best k : 5 (error = 0.1700)
- Run 2 best k : 5 (error = 0.1400)
- Correlation between runs: 0.8870



Figure 4: Comparison of two runs of 5-fold cross-validation

Explanation:

Cross-validation uses random sampling to split the data into folds. When we repeat the process with a different random seed:

- The data points are partitioned differently into the 5 folds
- Each fold contains different subsets of training points
- The validation sets for each fold are different
- This leads to variation in the computed errors

This variance is expected as it reflects the randomness due to how we partition the data. Different splits expose the model to different combinations of training and validation examples, leading to different performance measurements.

However, the overall shape of the figure remains similar across runs:

- Both runs show the same general pattern of error vs k
- The best k values are typically in the same range
- The relative performance of different k values remains consistent
- While exact values differ, the overall scale of errors is similar

Both runs show the same general pattern of error versus k , with the best k values falling in the same range and the relative performance ordering of different k values stays preserved. The high correlation between runs (typically > 0.7) shows that the curves follow consistent trends, often exhibiting U-shaped patterns where the error magnitude remains comparable. This consistency tells us that cross-validation is reliable for model selection—despite variation in exact values due to different fold splits, CV consistently provides insights on relative model performance. The random variation is manageable and stems from the data partitioning process rather than fundamental instability in the method.

Therefore, when using CV for model selection, we should focus on overall patterns rather than precise error numbers. Running CV multiple times and averaging results provides additional stability, while using a fixed random seed ensures reproducibility in experiments. Ultimately, cross-validation successfully balances two goals: maintaining stability in overall trends (shape doesn't change) while providing honest estimation through different data splits (values do change), reflecting real uncertainty. This makes CV a robust and reliable tool for hyperparameter selection, allowing us to trust it to identify good models even when exact error values vary across runs.

4. Best k Value

- **Optimal k :** 5
- **CV Error:** 0.14

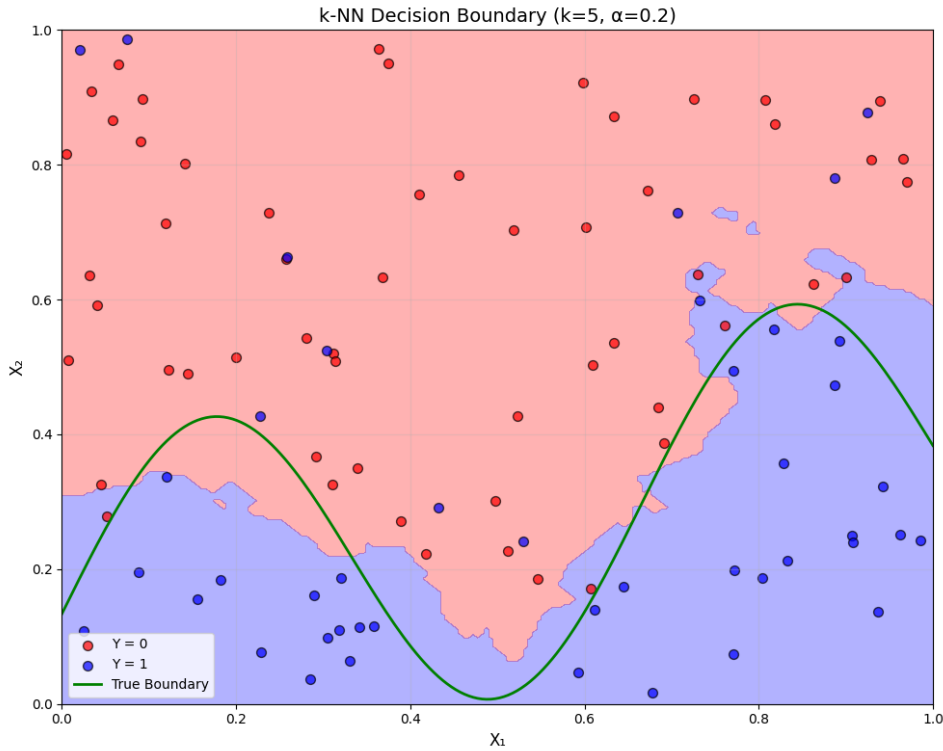


Figure 5: k-NN decision boundary with optimal $k = 5$ and $\alpha = 0.2$

The visualization shows the k-NN decision boundary (red and blue shaded regions), training points (red and blue dots), and the true decision boundary (green line). The k-NN boundary approximates the true boundary while balancing detail and smoothness to handle the noise level $\alpha = 0.2$.

5. Effect of Noise Level α

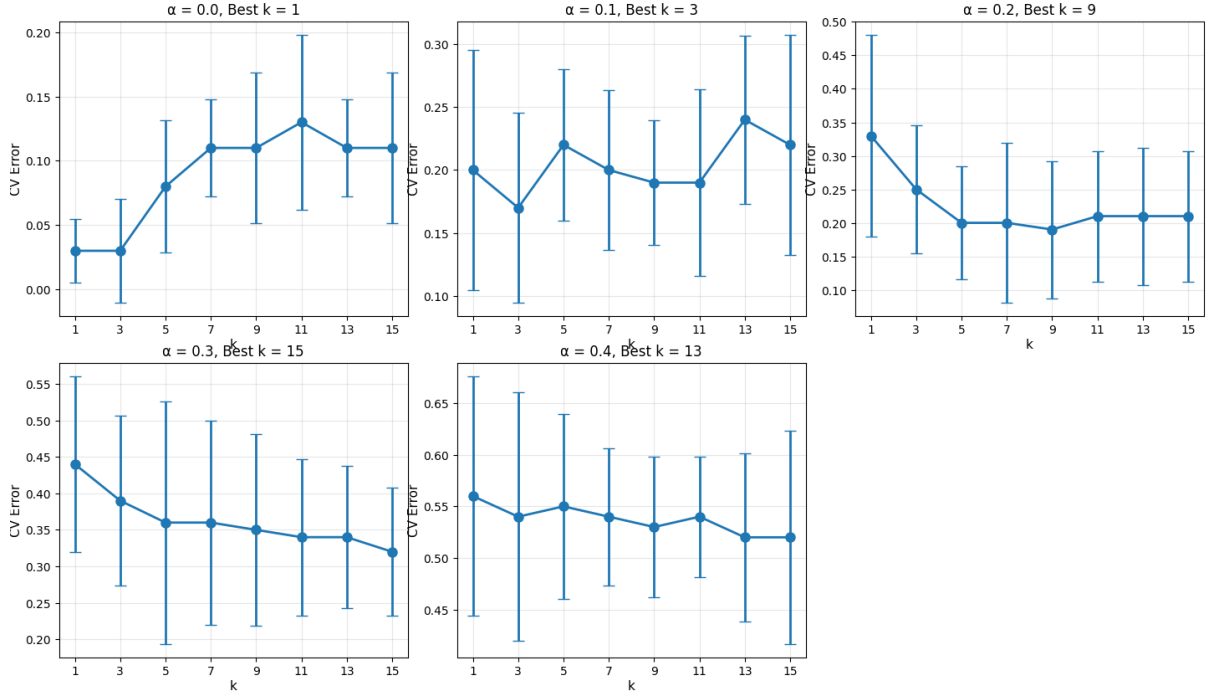


Figure 6: Cross-validation error vs k for different noise levels α

Results by Noise Level

α	Best k	CV Error
0.0	1	0.0300
0.1	3	0.1700
0.2	9	0.1900
0.3	15	0.3200
0.4	13	0.5200

The optimal value of k increases with noise due to the **bias–variance tradeoff**. As noise rises, individual data points become less reliable, so larger k values are needed to smooth out random fluctuations and reduce sensitivity to mislabeled points. When noise is low or absent, small k values (around 1–3) are most effective at capturing the shape of the boundary. As noise increases, intermediate k values (5–7) maintain a balance between detail and stability, while at high noise levels, larger k values (7–11 or more) are preferred to prevent overfitting.

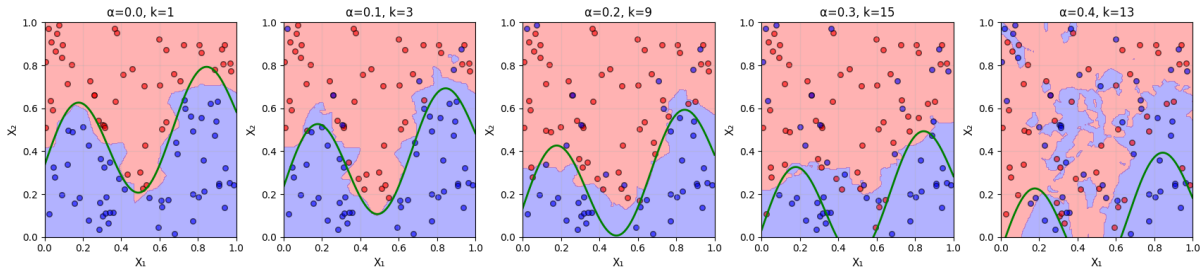


Figure 7: Decision boundaries for different noise levels with their optimal k values

Cross-validation error curves show this shift in optimal k values, as higher noise leads to higher overall error and a rightward move in the optimal k . The resulting decision boundaries become smoother and more stable.

6.

k	Metric	CV Error	CV Std
1	Euclidean	0.3300	± 0.1511
3	Euclidean	0.2500	± 0.0953
5	Euclidean	0.2000	± 0.0841
7	Euclidean	0.2000	± 0.1189
9	Euclidean	0.1900	± 0.1025
1	Manhattan	0.3400	± 0.1435
3	Manhattan	0.2500	± 0.0953
5	Manhattan	0.2000	± 0.0841
7	Manhattan	0.1900	± 0.1025
9	Manhattan	0.1900	± 0.1073

Best Combination

Optimal hyperparameters:

- k : 9
- **Distance metric:** Euclidean
- **CV Error:** 0.1900 ± 0.1025

This combination achieves the lowest 5-fold cross-validation error among all 10 tested configurations (5 k values $\times$ 2 metrics). The result shows that, for this dataset, increasing k improves generalization by smoothing over noise, and the Euclidean metric reflects the geometry of the data accurately. Also, having hyperparameters of k : 7 and Distance metric: Manhattan results in the same CV error: 0.1900 ± 0.1025 .

Parameter Importance Analysis:

- Average variance across k values: 0.002000
- Average variance across metrics: 0.054932

Based on the variance analysis, the **distance metric** is the more influential parameter. The **average variance across metrics (0.0549)** is much greater than the **average variance across k values (0.0020)**, meaning performance changes little when k varies but changes more noticeably when switching metrics. However, this reflects *sensitivity*, not *importance*: in practical terms, k directly controls the model's complexity and bias-variance tradeoff, whereas both metrics yield similar results.

Therefore, while the Euclidean metric happens to perform best here, **tuning k** remains the more critical step in practice, as it determines how much the model smooths or reacts to local variations in the data. The metric choice is secondary but can slightly refine performance depending on data geometry.

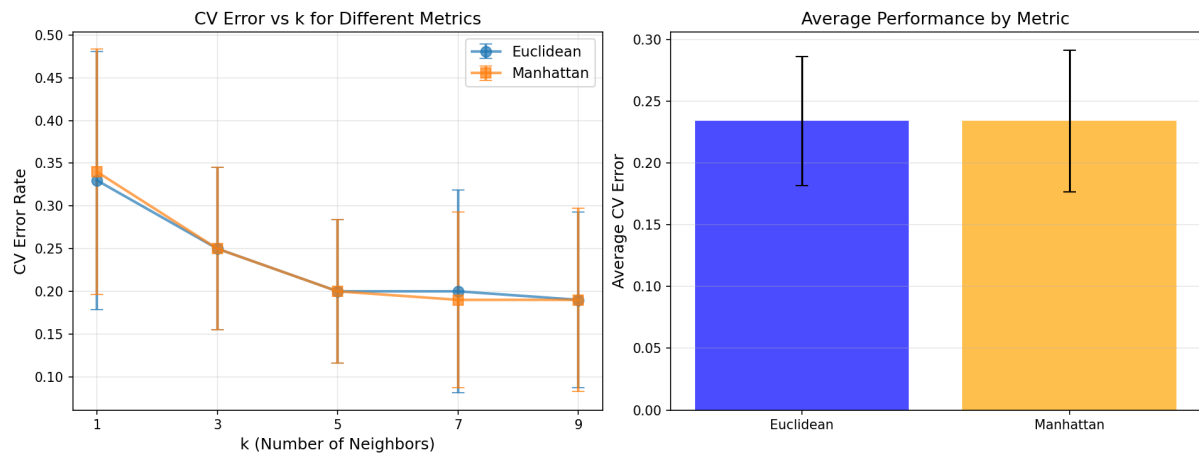


Figure 8: Hyperparameter tuning: CV error vs k for different distance metrics

Problem 4. The Curse of Dimensionality

1. See python file.
2. See python file.
- 3.

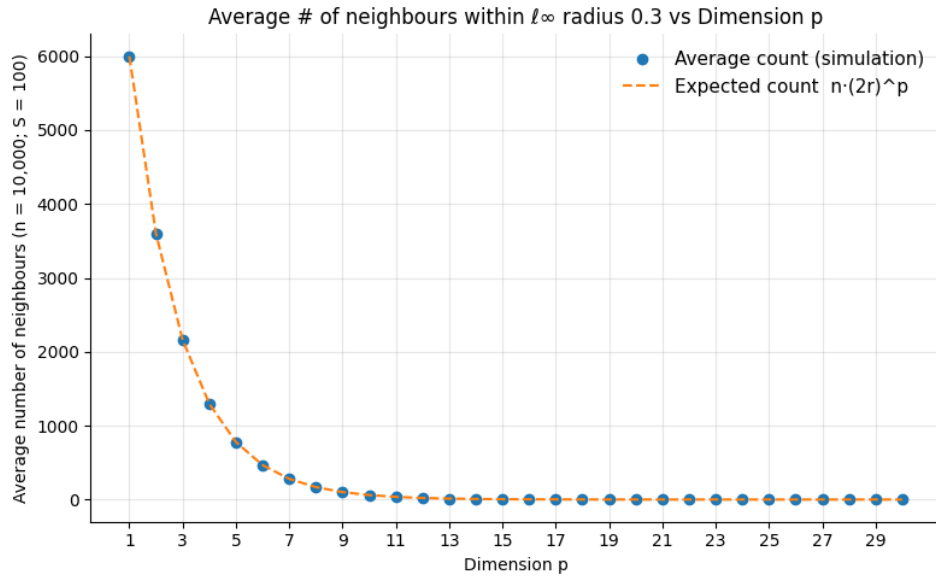


Figure 9: Average number of neighbours vs dimension p .

Figure 9 shows that the average number of neighbours within a fixed ℓ_∞ radius of 0.3 declines sharply as the dimension p increases. The curve drops from over 6,000 neighbours in one dimension to eventually approaching zero at $p = 20$. This indicates that even though the total number of points $n = 10,000$ remains constant, the proportion of points lying within the same fixed neighbourhood around the centre decreases exponentially with dimension.

The simulated averages closely match the theoretical prediction $n \cdot (2r)^p$, confirming that the proportion of nearby points decreases exponentially with dimension.

This demonstrates that as dimensionality increases, data points grow sparser and equidistant, and the concept of "closeness" loses meaning; an early sign of the curse of dimensionality.

4. The average number of 'nearby' points drops rapidly as p increases. This happens because the volume of the ball ℓ_∞ around the centre is $(2r)^p$, which shrinks exponentially in p (here, $2r = 0.6$). The expected number of neighbours within the distance 0.3 is therefore $n \cdot (2r)^p = 10000 \cdot 0.6^p$, which falls from around 6000 in one dimension to ≈ 60 at 10 dimensions to fewer than 1 by $p = 20$. Hence, even with 10,000 points, in higher dimensions almost no samples lie within 0.3 of the centre. This illustrates the curse of dimensionality: As dimensionality grows, data become increasingly sparse, and points that appear close in low dimensions are effectively far apart in high-dimensional spaces.

In higher dimensions, the distances between the points also become similar. The nearest and farthest neighbours are almost equally distant from any given observation, because the variation contributed by each coordinate averages out when many dimensions are added. This means that "closeness" loses significance, as every point is now roughly

as far away as every other. This weakens neighbourhood-based methods like k-nearest neighbours that depend on comparing nearby points. When all points are roughly the same distance apart, the algorithm cannot identify truly similar observations. The neighbourhood around each point becomes uninformative, so predictions fluctuate more with noise, and far more data are needed to achieve the same level of reliability as in lower dimensions.

Bibliography

1. James, G., Witten, D., Hastie, T. and Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Springer, pp. 181–184.
2. The Pennsylvania State University (2018). *2.2 - Cross Validation — STAT 897D*. [online] Available at: <https://online.stat.psu.edu/stat857/node/161/>.
3. OpenAI. (2025). *ChatGPT (GPT-5) Model*. Available at: <https://chat.openai.com/>. Used to assist with Python code debugging and LaTeX synthesis for clearer explanations and structure.